Lab Notebook: Frequency Counter Experiment[†]

Pritam Ghosh

February 26, 2025

1 Objective

To design and implement a frequency counter using a combination of digital ICs, an Arduino, and a display unit to measure and analyze frequency.

2 IC 7476

2.1 Introduction

The IC 7476 is a **Dual J-K Flip-Flop** integrated circuit, widely used in digital electronics for memory storage and sequential logic applications. This document provides detailed specifications, features, pin configurations, and the operating principle of the IC 7476.

2.2 Key Features

- Contains **two independent J-K flip-flops** in a single IC.
- Each flip-flop has:
  - Separate **J** and **K** inputs.
  - Individual **Clock**, **Preset**, and **Clear** inputs.
- Operates as a **master-slave flip-flop**, where data is latched on the positive edge of the clock and transferred on the negative edge.
- Outputs are compatible with CMOS, NMOS, and TTL logic families.
- Offers fast switching speeds and reliable operation.

2.3 Pin Configuration

The IC comes in a **16-pin DIP package** with the following pin layout:

[†]Use Digital copy for the colored plots and diagrams.

Pin Number	Description
1	Clock 1 Input
2	Preset 1 Input
3	Clear 1 Input
4	J1 Input
5	Vcc (Power Supply)
6	Clock 2 Input
7	Preset 2 Input
8	Clear 2 Input
9	J2 Input
10	Complement Q2 Output
11	Q2 Output
12	K2 Input
13	Ground
14	Complement Q1 Output
15	Q1 Output
16	K1 Input

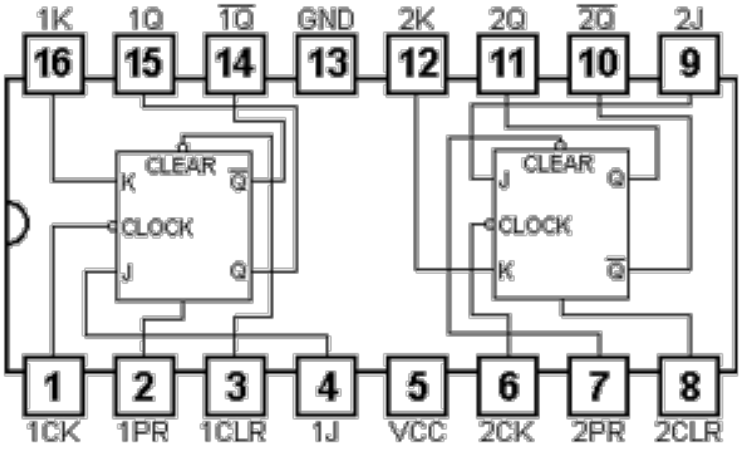


Figure 1: Diagram of IC 7476

2.4 Electrical Characteristics

- **Propagation Delay:**
  - LOW-to-HIGH: approximately 22 ns.
  - HIGH-to-LOW: approximately 15 ns.
- **Input Clamp Voltage:** -1.5 V.
- **Short Circuit Output Current:** -18 to -55 mA.

2.5 Applications

- Memory elements in sequential circuits.
- Frequency dividers.
- Counters and shift registers.
- Data storage in microcontroller-based systems.

2.6 Operating Principle

The J-K flip-flop operates based on its truth table:

- When J=K = HIGH J=K=HIGH, the output toggles on every clock pulse.
- When J = HIGH J=HIGH and K = LOW K=LOW, the output is set to HIGH.
- When J = LOW J=LOW and K = HIGH K=HIGH, the output is reset to LOW.
- When J = K = LOW J=K=LOW, the output remains unchanged.

3 4-Bit Asynchronous Up Counter using IC 7476

3.1 Components Required:

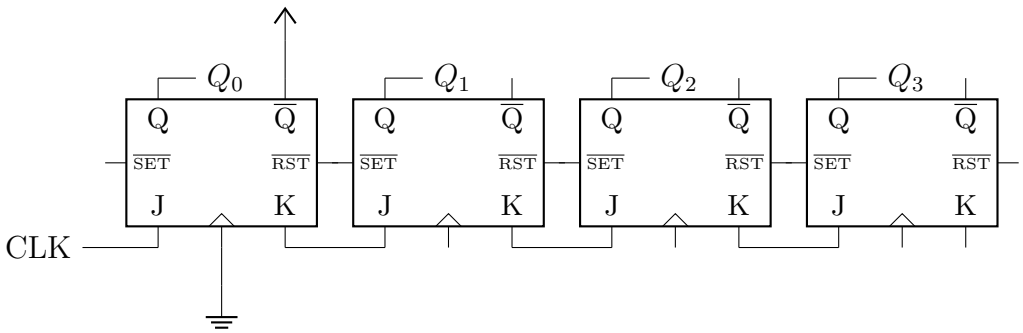
- IC 7476 (JK Flip-Flop)
- Breadboard
- Connecting wires
- Resistors
- LEDs (to visualize count)
- Power supply (5V DC)
- Square wave generator (for clock pulse)

Procedure:

1. Connect the IC 7476 on the breadboard.
2. Wire the JK Flip-Flop in asynchronous mode to form a 4-bit counter.
3. Connect LEDs to the output to observe the counting sequence.
4. Provide a clock pulse and verify the up-counting operation.

3.2 Working Principle

3.2.1 Circuit Diagram



1. **Circuit Configuration**:
   - 4 D-type flip-flops (or JK flip-flops in toggle mode) connected in series
   - Clock input applied to the first flip-flop (LSB: Q_0)
   - Subsequent flip-flop clocks triggered by **complemented output ($\bar{Q}$)** of previous stage
   - Preset (PR) and Clear (CLR) inputs tied to logic HIGH
2. **Toggle Operation**:
   - Flip-flops operate in **toggle mode** (output inverts on every clock pulse)
   - Q_0 toggles with each clock edge
   - Q_1 toggles on Q_0 HIGH→LOW transition
   - Cascaded triggering continues through Q_2 and Q_3
3. **Ripple Effect**:
   - Propagation delays cause staggered counting
   - Temporary glitches during state transitions (e.g., 0111→1000)

3.2.2 Truth Table

Clock Pulse	Q_3 **(MSB)**	Q_2	Q_1	Q_0 **(LSB)**	**Decimal Equivalent**
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	2
3	0	0	1	1	3
4	0	1	0	0	4
5	0	1	0	1	5
6	0	1	1	0	6
7	0	1	1	1	7
8	1	0	0	0	8
9	1	0	0	1	9

10	1	0	1	0	10
11	1	0	1	1	11
12	1	1	0	0	12
13	1	1	0	1	13
14	1	1	1	0	14
15	1	1	1	1	15
16	0	0	0	0	0

3.2.3 Key Characteristics

- **Modulus**: MOD-16 (16 states: 0000-1111)
- **Frequency Division**:

$$f_{Q0} = \frac{f_{CLK}}{2}$$
$$f_{Q1} = \frac{f_{CLK}}{4}$$
$$f_{Q2} = \frac{f_{CLK}}{8}$$
$$f_{Q3} = \frac{f_{CLK}}{16}$$

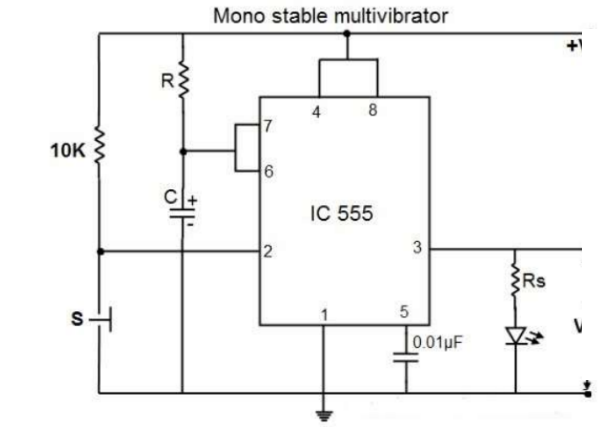
- Propagation delay accumulates with each stage
- Maximum operating frequency: $f_{max} = \frac{1}{n \times t_p}$

3.3 Error Analysis

- Propagation delay in JK Flip-Flops may affect counting accuracy.
- Oscilloscope measurement resolution affects pulse width detection.
- Power fluctuations may impact circuit stability.
- Human errors in wiring and component placements.

4 Monostable multivibrator using 555 timer

Circuit Diagram:



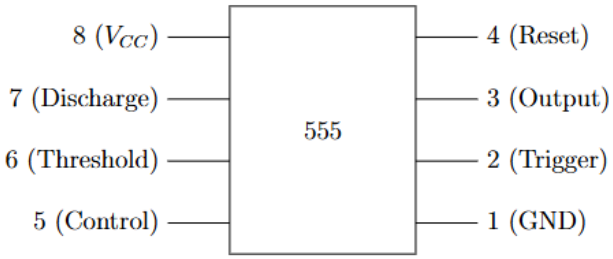


Figure 2: Pin Configuration

4.1 Working:

When the trigger input is slightly less than $+V_{CC}/3$, the comparator has a high output and resets the flip-flop. This cuts off the transistor which allows the capacitor to charge.

When the capacitor voltage is slightly greater than $+2V_{CC}/3$, the upper comparator has a high output which sets the flip-flop.

The figure below shows typical waveforms.

The trigger input is a narrow pulse with a quiescent value of $+V_{CC}$. The pulse must drop below $+V_{CC}/3$ to reset the flip-flop and allow the capacitor to charge.

When the threshold voltage slightly exceeds $+2V_{CC}/3$, the flip-flop sets which saturates the transistor and discharges the capacitor.

This results in a rectangular output pulse.

The capacitor C has to charge through the resistance R. The larger the RC time constant, the longer it takes for the capacitor voltage to reach $+2V_{CC}/3$.

The RC time constant controls the width of the output pulse.

Consider the capacitor charging equation:

$$v(t) = v_f + (v_i - v_f)(1 - e^{-t/RC})$$

Here: - $v_i = 0$, - $v_f = V_{CC}$, - At $t = W$, $v(t) = +2V_{CC}/3$.

Substituting these values in the above equation:

$$\frac{2V_{CC}}{3} = V_{CC}(1 - e^{-W/RC})$$

Simplifying:

$$e^{-W/RC} = 1 - \frac{2}{3} = \frac{1}{3}$$

Taking natural logarithms:

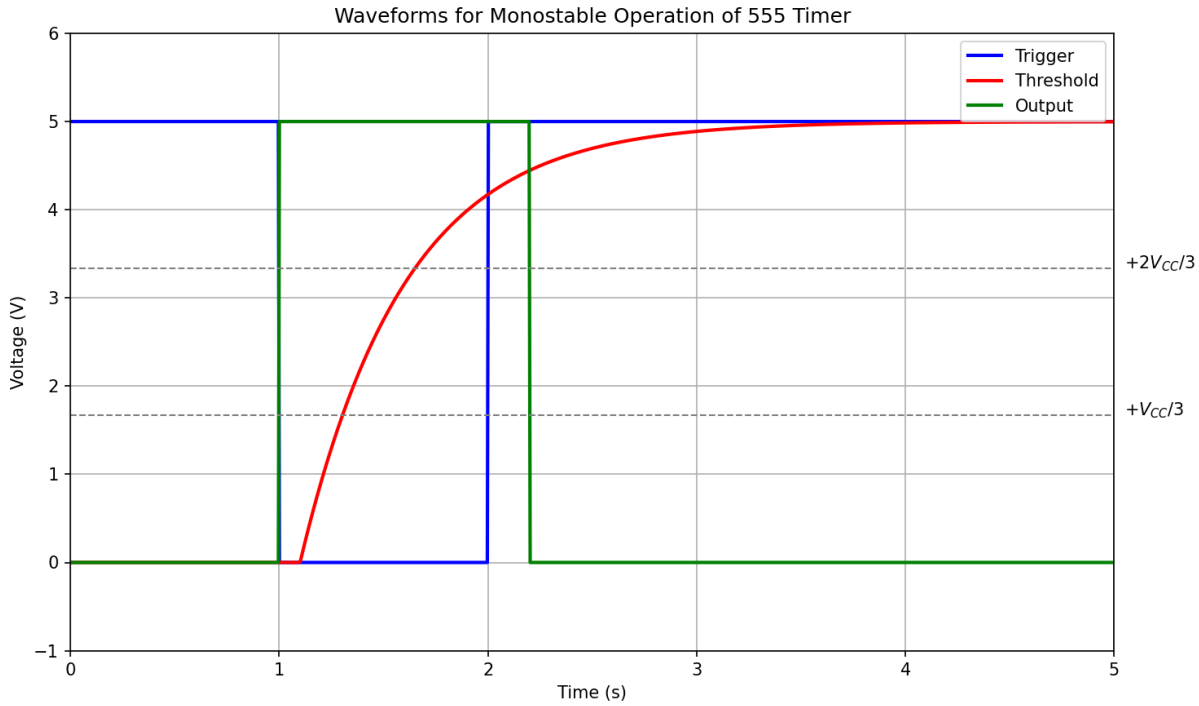
$$W = 1.0986RC = 1.1RC$$

The pin 5 (control pin) is bypassed to ground through a small capacitor, typically $0.01\,\mu F$. This provides noise filtering for the control voltage.

Grounding pin 4 inhibits the timer. To avoid accidental reset, pin 4 is usually connected to the supply voltage as shown in the figure.

4.2 Waveform Plot for Monostable Operation of 555 Timer

The figure below shows the typical waveforms for the monostable operation of a 555 timer.



4.3 Observation

Applying the trigger we can see the LED light remains turn on for 1 Second.

5 2-Digit Counter using CD4026 with 7-Segment Display

Components Required:

- CD4026 (7-segment counter driver IC)
- 7-segment display
- Clock signal generator
- Resistors, breadboard, and wires

Procedure:

1. Connect CD4026 to the 7-segment display as per the datasheet.

2. Apply a clock pulse to the counter input.
3. Observe the display incrementing on each pulse.

Circuit Diagram:

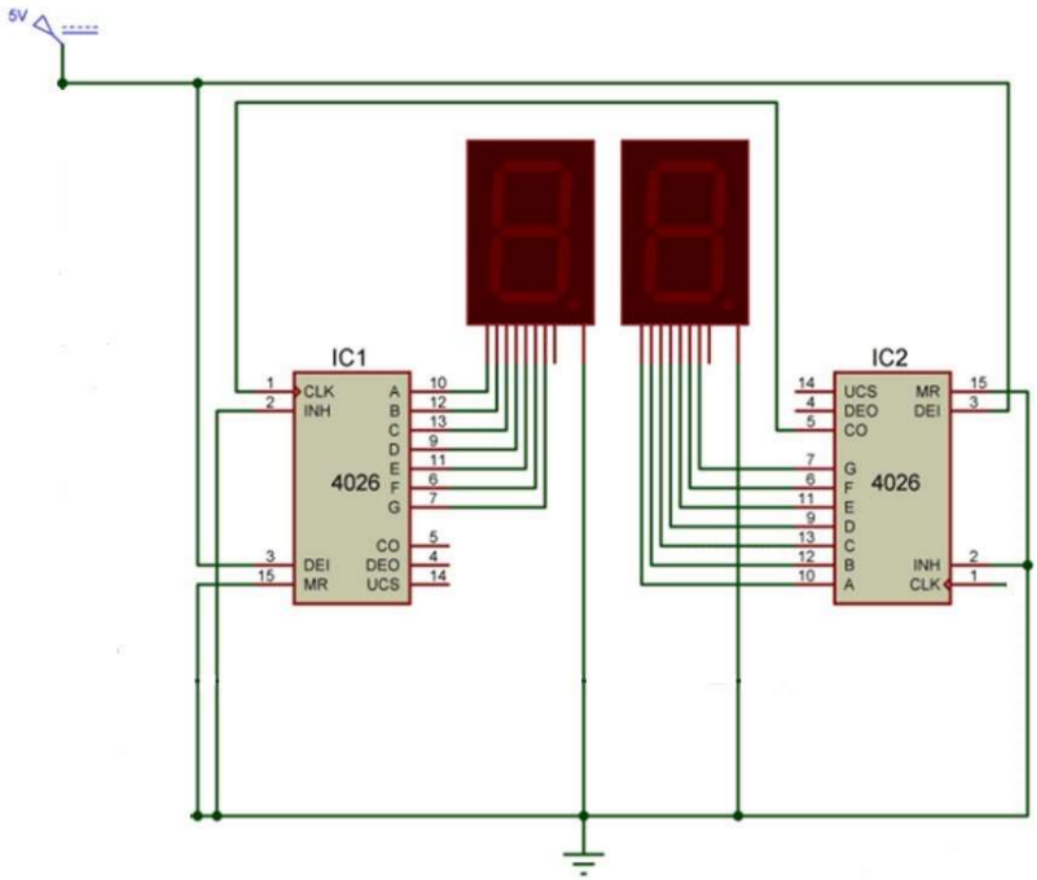


Figure 3: Circuit diagram of 2 digit Counter

Key connections:

- IC1 (units place) CLK pin → Clock source
- IC1 CO pin (carry-out) → IC2 CLK pin
- 7-segment displays connected to respective IC outputs [a-g]
- Common cathode pins grounded
- MR (pin 15) connected to reset switch

5.1 Working Principle

$$\text{Count sequence} = (\text{IC2 count} \times 10) + \text{IC1 count} \tag{1}$$

- Each clock pulse increments IC1 (0-9)
- When IC1 resets from 9→0, CO pin triggers IC2

- IC2 increments tens place (0-9)
- Display enables (pin 3) held high for continuous display

5.2 Features

Parameter	Value
Counting Range	00-99
Max Clock Frequency	6MHz
Supply Voltage	3-18V DC
Display Type	7 segment decade counter

5.3 Applications

- Digital visitor counters
- Event counting systems
- Basic instrumentation displays

6 Pulse Counter Circuit

6.1 Working Principle

The pulse generator produces pulses that are fed to one input of the AND gate. The monostable multivibrator provides a 1-second pulse to the other input of the AND gate. The AND gate will only output pulses when both inputs are HIGH, effectively counting pulses per second.

6.2 Output Buffer Circuit

The output from the AND gate is connected to a transistor buffer circuit to boost the current sufficiently to drive the 2-digit counter circuit (CD4026). The buffer circuit consists of:

- Input resistor (R_1)
- Pull-up resistor (R_2)
- NPN transistor

The output of this buffer circuit connects directly to the clock (CLK) input of the CD4026 counter IC.

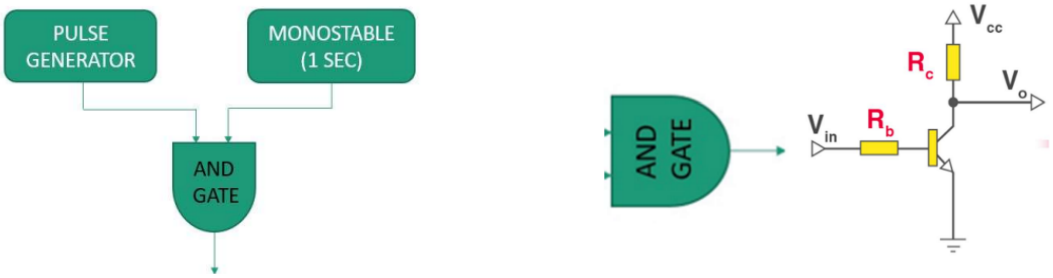


Figure 4: This Circuit output goes to the clock of the 2-digit counter circuit

6.3 Observations

- The AND gate successfully gates the pulse generator output
- The transistor buffer provides sufficient current gain to drive the CD4026
- The counter correctly increments with each pulse during the 1-second window

6.4 Calculations

For the transistor buffer circuit:

$$I_{\text{base}} = \frac{V_{\text{in}} - 0.7V}{R_1}$$

$$I_{\text{collector}} = \beta \times I_{\text{base}}$$

Where β is the current gain of the transistor.

6.5 Conclusion

The circuit successfully counts pulses per second using the AND gate configuration and provides reliable clock pulses to the CD4026 counter IC. The transistor buffer effectively boosts the current to drive the counter circuit.

7 Digital Tachometer for Fan RPM Measurement

7.1 Objective

To design and implement an Arduino-based system that can measure the RPM (Revolutions Per Minute) of a fan using an IR sensor, display the readings on an LCD screen, and compare these measurements with an oscilloscope.

7.2 Components Required

- Arduino Board
- IR Sensor Module
- LCD Screen (compatible with Hitachi HD44780 driver)
- Pin headers for LCD connections
- 10k ohm potentiometer (for LCD contrast adjustment)
- 220 ohm resistor
- Hook-up wires
- Breadboard
- Oscilloscope (for verification)
- Fan (to be measured)

7.3 Circuit Connections

7.3.1 LCD to Arduino Connections

- LCD Pins 1, 3, 5, 16 $\rightarrow$ GND
- LCD Pins 2, 15 $\rightarrow$ VCC (+5V)
- LCD Pin 4 $\rightarrow$ Arduino pin D7 (RS)
- LCD Pin 6 $\rightarrow$ Arduino pin D6 (E)
- LCD Pin 11 $\rightarrow$ Arduino pin D5 (D4)
- LCD Pin 12 $\rightarrow$ Arduino pin D4 (D5)
- LCD Pin 13 $\rightarrow$ Arduino pin D6 (D6)
- LCD Pin 14 $\rightarrow$ Arduino pin D7 (D7)

7.3.2 IR Sensor to Arduino Connections

- IR Sensor Module Pin GND → GND
- IR Sensor Pin VCC → VCC
- IR Sensor Pin OUT → Arduino Pin A0

7.4 Circuit Diagram Notes

- The 10k potentiometer is connected to the LCD for contrast adjustment
- The 220 ohm resistor is likely used for the LCD backlight or as a current limiter for the IR sensor

7.5 Code Implementation

```
#include <LiquidCrystal.h>

// Initialize the LCD with the pins
LiquidCrystal lcd(7, 6, 5, 4, 6, 7);

// Variables for RPM calculation
const int sensorPin = A0;
volatile unsigned long pulseCount = 0;
unsigned long lastTime = 0;
unsigned long rpm = 0;
unsigned long updateInterval = 1000;  // Update display every second

void setup() {
  // Initialize serial communication for debugging
  Serial.begin(9600);

  // Set up the LCD
  lcd.begin(16, 2);  // 16 columns, 2 rows
  lcd.print("Fan RPM Meter");

  // Set up the sensor pin as input
  pinMode(sensorPin, INPUT);

  // Attach interrupt for pulse counting
  attachInterrupt(digitalPinToInterrupt(2), countPulse, RISING);

  delay(1000);
}

void loop() {
  // Calculate and update RPM every second
  if (millis() - lastTime >= updateInterval) {
```

```
    // Calculate RPM (pulses per minute)
    // Assuming 1 pulse per revolution
    rpm = (pulseCount * 60) / (updateInterval / 1000);

    // Reset pulse count
    pulseCount = 0;

    // Update display
    lcd.setCursor(0, 1);
    lcd.print("RPM: ");
    lcd.print(rpm);
    lcd.print("    ");  // Clear any remaining characters

    // Print to serial for oscilloscope comparison
    Serial.print("RPM: ");
    Serial.println(rpm);

    lastTime = millis();
  }
}

// Interrupt service routine for pulse counting
void countPulse() {
  pulseCount++;
}
```

7.6 Testing Procedure

1. Assemble the circuit according to the connection details
2. Upload the code to the Arduino board
3. Position the IR sensor to detect fan blade passages
4. Power on the system and observe the RPM readings on the LCD
5. Connect the oscilloscope to the IR sensor output (Pin A0)
6. Compare the frequency measured by the oscilloscope with the calculated RPM

7.7 Oscilloscope Comparison Method

- Measure the frequency (f) of pulses using the oscilloscope
- Calculate RPM using the formula: RPM = $f \times 60$
- Compare this value with the Arduino LCD display
- Note any discrepancies and calculate the percentage error

7.8 Expected Results

- The LCD should display stable RPM readings that update every second
- The oscilloscope frequency measurement should correlate with the Arduino calculation
- Some minor differences may occur due to timing precision and sensor sensitivity

7.9 Observations and Troubleshooting

- If readings are unstable, check the IR sensor positioning
- Adjust the 10k potentiometer to optimize LCD contrast
- Ensure proper grounding to minimize electrical noise
- For multi-blade fans, divide the pulse count by the number of blades to get accurate RPM

7.10 Conclusion

This setup provides a practical implementation of a digital tachometer using Arduino. The comparison with an oscilloscope serves as calibration and verification of the measurement accuracy.

Future Improvements:

- Implementing a higher-resolution counter for more precise frequency measurement.
- Using a microcontroller for real-time data logging.
- Developing a wireless display for RPM monitoring.